Chapter 10: Lambdas

By Irina Galata

A previous chapter taught you about functions. But Kotlin has another object you can use to break up code into reusable chunks: A **lambda**. These have many uses, and become particularly useful when dealing with collections such as an array or map.

A lambda expression is simply a function with no name; you can assign it to a variable and pass it around like any other value. This chapter shows you how convenient and useful lambdas can be.

Lambda basics

Lambdas are also known as **anonymous functions**, and derive their name from the **lambda calculus** of Alonzo Church, in which all functions are anonymous. Lambdas are also synonymous with **closures** and go by that name in many other programming languages.

Closures are so named because they have the ability to "close over" the variables and constants within the closure's own scope. This simply means that a lambda can access, store and manipulate the value of any variable or constant from the surrounding context, acting as a nested function. Variables and constants used within the body of a lambda are said to have been **captured** by the lambda.

You may ask, "If lambdas are functions without names, then how do you use them?" To use a lambda, you first have to assign it to a variable or constant, including as an argument to another function.

Here's a declaration of a variable that can hold a lambda:

```
var multiplyLambda: (Int, Int) -> Int
```

`multiplyLambda` takes two `Int` values and returns an `Int`. Notice that this is exactly the same as a variable declaration for a function. As was said, a lambda is simply a function without a name. The type of a lambda is a function type.

You assign a lambda to a variable like so:

```
multiplyLambda = { a: Int, b: Int -> Int
  a * b
}
```

This looks similar to a function declaration, but there are subtle differences. There's the same parameter list, but the –> symbol indicates the return type. The body of the lambda begins after the return type. The lambda expression returns the value of the last expression in the body.

With your lambda variable defined, you can use it just as if it were a function, like so:

```
val lambdaResult = multiplyLambda(4, 2) // 8
```

As you'd expect, `result` equals 8. Again, though, there's a subtle difference. A lambda does not allow the use of names for arguments; for instance, you can't write `multiplyLambda(a = 4, b = 2)`. Unlike functions, you can't use the parameter names for labeling the arguments.

Shorthand syntax

Compared to functions, lambdas are designed to be lightweight. There are many ways to shorten their syntax. First, you can use Kotlin's type inference to shorten the syntax by removing the type information:

```
multiplyLambda = { a, b ->
  a * b
}
```

Remember, you already declared `multiplyLambda` as a lambda taking two `Int`s and returning an `Int`, so you can let Kotlin infer these types for you.

it keyword

For a lambda that has only one parameter, you can shorten it even further using the `it` keyword. As an example, look at this lambda:

```
var doubleLambda = { a: Int ->
  2 * a
}
```

Since there is only one parameter and the lambda type is now specified, the lambda can be shortened to:

```
doubleLambda = { 2 * it }
```

You can also use `it` in a new declaration:

```
val square: (Int) -> Int = { it * it }
```

Lambdas as arguments

Consider the following code:

```
fun operateOnNumbers(
  a: Int,
  b: Int,
  operation: (Int, Int) -> Int
): Int {
  val result = operation(a, b)
  println(result)
  return result
}
```

This declares a function named `operateOnNumbers`, which takes `Int` values as its first two parameters. The third parameter is named `operation` and is of a function type. `operateOnNumbers` itself returns an `Int`.

You can then use `operateOnNumbers` with a lambda, like so:

```
val addLambda = { a: Int, b: Int ->
  a + b
}
operateOnNumbers(4, 2, operation = addLambda) // 6
```

Remember, lambdas are simply functions without names. So you shouldn't be surprised to learn that you can also pass in a function as the third parameter of `operateOnNumbers`, like so:

```
fun addFunction(a: Int, b:Int) = a + b

operateOnNumbers(4, 2, operation = ::addFunction) // 6
```

`operateOnNumbers` is called the same way, whether the `operation` is a function or a lambda. The `::` operator is the *reference operator*; in this case, it instructs the program to find `addFunction` in the current scope.

The power of the lambda syntax comes in handy again.

You can define the lambda inline with the `operateOnNumbers` function call, like this:

```
operateOnNumbers(4, 2, operation = { a: Int, b: Int ->
  a + b
})
```

There's no need to define the lambda and assign it to a local variable or constant. You can simply declare the lambda right where you pass it into the function as an argument!

But recall that you can simplify the lambda syntax to remove a lot of the boilerplate code. You can therefore reduce the above to the following:

```
operateOnNumbers(4, 2, { a, b ->
  a + b
})
```

In fact, you can even go a step further. The + operator is just an operator function `plus()` in the `Int` class that takes two arguments and returns one result so you can write:

```
operateOnNumbers(4, 2, operation = Int::plus)
```

There's one more way you can simplify the syntax, but it can only be done when the lambda is the final argument passed to a function. In this case, you can move the lambda outside of the function call:

```
operateOnNumbers(4, 2) { a, b ->
  a + b
}
```

This may look strange, but it's just the same as the previous code snippet using the lambda syntax, except you've removed the `operation` label and pulled the braces outside of the function call parameter list. This is called **trailing lambda syntax**.

Lambdas with no meaningful return value

Until now, all the lambdas you've seen have taken one or more parameters and have returned values. But just like functions, lambdas aren't required to do these things. A lambda will always return the value of its last expression, so here is how you define a lambda that takes no parameters and returns only the `Unit` object:

```
var unitLambda: () -> Unit = {
  println("Kotlin Apprentice is awesome!")
}
unitLambda()
```

The lambda's type is `() -> Unit`. The empty parentheses denote there are no parameters. You must declare a return type, so Kotlin knows you're declaring a lambda. This is where `Unit` comes in handy, when the lambda needs to return no meaningful value.

If you literally want the lambda to not return a value, you must use the `Nothing` type, like so:

```
var nothingLambda: () -> Nothing = {
  throw NullPointerException()
}
```

Since an exception is thrown, the lambda does not actually return a value.

Capturing from the enclosing scope

Let's return to an important characteristic of lambdas, as they act as closures: they can access the variables and constants from within their own scope.

> **Note**: Recall that scope defines the range in which an entity (variable, constant, etc) is accessible. You saw a new scope introduced with `if` statements. Lambdas also introduce a new scope and inherit all entities visible to the scope in which they are defined.

For example, take the following lambda:

```
var counter = 0
val incrementCounter = {
  counter += 1
}
```

`incrementCounter` is rather simple: It increments the `counter` variable. The `counter` variable is defined outside of the lambda. The lambda is able to access the variable because the lambda is defined in the same scope as the variable. The lambda is said to **capture** the `counter` variable. Any changes it makes to the variable are visible both inside and outside the lambda.

Let's say you call the lambda five times, like so:

```
incrementCounter()
incrementCounter()
incrementCounter()
incrementCounter()
incrementCounter()
```

After these five calls, `counter` will equal 5.

The fact that lambdas can be used to capture variables from the enclosing scope can be extremely useful. For example, you could write the following function:

```
fun countingLambda(): () -> Int {
  var counter = 0
  val incrementCounter: () -> Int = {
    counter += 1
    counter
  }
  return incrementCounter
}
```

This function takes no parameters and returns a lambda. The lambda it returns takes no parameters and returns an `Int`.

The lambda returned from this function will increment its internal counter each time it is called. Each time you call this function you get a different counter.

For example, this could be used like so:

```
val counter1 = countingLambda()
val counter2 = countingLambda()

println(counter1()) // > 1
println(counter2()) // > 1
println(counter1()) // > 2
println(counter1()) // > 3
println(counter2()) // > 2
```

The two counters created by the function are mutually exclusive and count independently. Neat!

Custom sorting with lambdas

Lambdas come in handy when you start looking deeper at collections. In Chapter 8, you used array's `sort` method to sort an array. By specifying a lambda, you can customize how things are sorted.

You call `sorted()` to get a sorted version of the array like so:

```
val names = arrayOf("ZZZZZZ", "BB", "A", "CCCC", "EEEEE")
names.sorted() // A, BB, CCCC, EEEEE, ZZZZZZ
```

By specifying a custom lambda passed to `compareBy()`, which returns a `Comparator` for `sortedWith()`, you can change the details of how the array is sorted.

Specify a trailing lambda for `compareBy()` like so:

```
val namesByLength = names.sortedWith(compareBy {
  -it.length
})
println(namesByLength) // > [ZZZZZZ, EEEEE, CCCC, BB, A]
```

Now the array is sorted by the length of the string with longer strings coming first. The minus sign causes the sort to be descending by length.

Iterating over collections with lambdas

In Kotlin, collections implement some very handy features often associated with **functional programming**. These features come in the shape of functions that you can apply to a collection to perform an operation on it.

Operations include things like transforming each element or filtering out certain elements. These functions make use of lambdas.

The first of these functions, `forEach`, lets you loop over the elements in a collection and perform an operation like so:

```
val values = listOf(1, 2, 3, 4, 5, 6)
values.forEach {
  println("$it: ${it * it}")
}
// > 1: 1
// > 2: 4
// > 3: 9
// > 4: 16
// > 5: 25
// > 6: 36
```

This loops through each item in the collection printing the value and its square.

Another function allows you to filter out certain elements:

```
var prices = listOf(1.5, 10.0, 4.99, 2.30, 8.19)

val largePrices = prices.filter {
  it > 5.0
}
```

Here, you create a list of `Double` to represent the prices of items in a shop. To filter out the prices which are greater than $5, you use the `filter` function. This function looks like so:

```
public inline fun <T> Iterable<T>.filter(predicate: (T) ->
Boolean): List<T>
```

This means that `filter` takes a single parameter named `predicate`, which is a lambda (or function) that takes a `T` and returns a `Boolean`. The `filter` function then returns a list of `T`. In this context, `T` refers to the type of items in the list. In the example above, `Double`.

Don't worry if you don't understand the purpose of `T` here. You'll learn more about this in Chapter 18, "Generics."

The lambda's job for `filter` is to return `true` or `false` depending on whether or not the value should be kept or not. The list returned from `filter` will contain all elements for which the lambda returned `true`.

In the example, `largePrices` will contain:

```
[10.0, 8.19]
```

> **Note:** The array that is returned from `filter` (and all of these functions) is a new array. The original is not modified at all.

However, there is more!

Imagine you're having a sale and want to discount all items to 90% of their original price. There's a handy function named `map` which can achieve this:

```
val salePrices = prices.map {
  it * 0.9
}
```

The `map` function will take a lambda, execute it on each item in the list and return a new list containing each result with the order maintained. In this case, `salePrices` will contain:

```
[1.35, 9.0, 4.4910000000000005, 2.07, 7.3709999999999996]
```

> **Note**: Be sure not to confuse the `map` function used to transform collections with the various `Map` types such as `HashMap` or functions like `mapOf` that create map objects.

The `map` function can also be used to change the type. You can do that like so:

```
val userInput = listOf("0", "11", "haha", "42")
val numbers = userInput.map {
  it.toIntOrNull()
}
println(numbers) // > [0, 11, null, 42]
```

This takes some strings that the user input and turns them into an array of `Int?`. They need to be nullable because the conversion from `String` to `Int` might fail.

If you want to filter out the invalid, `null`, values, you can use `mapNotNull()` like so:

```
val numbers2 = userInput.mapNotNull {
  it.toIntOrNull()
}
println(numbers2) // > [0, 11, 42]
```

This is almost the same as `map` except it tosses out the `null` values.

Another handy function is `fold`, which takes a starting value and a lambda. The lambda takes two values: the current value and an element from the list. The lambda returns the next value that should be passed into the lambda as the current value parameter.

This could be used with the `prices` list to calculate the total, like so:

```
var sum = prices.fold(0.0) { a, b ->
  a + b
}
```

The initial value is 0.0. Then the lambda calculates the sum of the current value plus the current iteration's value. Thus you calculate the total of all the values in the array. In this case, `sum` will be:

```
println(sum) // > 26.980000000000004
```

A function closely related to `fold` is `reduce`. In Kotlin, `reduce` uses the first element in the collection as the starting value:

```
sum = prices.reduce { a, b ->
  a + b
}
println(sum) // > 26.980000000000004
```

Now that you've seen `filter`, `map`, `fold`, and `reduce`, hopefully it's becoming clear how powerful these functions can be, especially thanks to the syntax of lambdas. In just a few lines of code, you have calculated some rather complex values from the collection.

Many of these functions can also be used with maps. Imagine you represent the stock in your shop by a dictionary mapping the price to number of items at that price. You could use that to calculate the total value of your stock like so:

```
val stock = mapOf(
  1.5 to 5,
  10.0 to 2,
```

```
    4.99 to 20,
    2.30 to 5,
    8.19 to 30
)
var stockSum = 0.0
stock.forEach {
    stockSum += it.key * it.value
}
```

In this case, the parameter to the `forEach` function is a `Map.Entry` containing the `key` and `value` from the map elements.

Here, the result is:

```
println(stockSum) // > 384.5
```

That wraps up collection iteration with lambdas!

Mini-exercises

1. Create a constant list called `nameList` which contains some names as strings. Any names will do — make sure there's more than three. Now use `fold` to create a string which is the concatenation of each name in the list.
2. Using the same `nameList` list, first filter the list to contain only names which have more than four characters in them, and then create the same concatenation of names as in the above exercise.

> **Hint**: you can chain these operations together.

3. Create a constant map called `namesAndAges` which contains some names as strings mapped to ages as integers. Now use `filter` to create a map containing only people under the age of 18.
4. Using the same `namesAndAges` map, filter out the adults (those 18 or older) and then use `map` to convert to a list containing just the names (i.e., drop the ages).

Challenges

Check out the challenges below to test your knowledge of Kotlin lambdas.

Challenge 1: Repeating yourself

Your first challenge is to write a function that will run a given lambda a given number of times.

Declare the function like so:

```
fun repeatTask(times: Int, task: () -> Unit)
```

The function should run the `task` lambda `times` number of times.

Use this function to print `"Kotlin Apprentice is a great book!"` 10 times.

Challenge 2: Lambda sums

In this challenge, you're going to write a function that you can reuse to create different mathematical sums.

Declare the function like so:

```
fun mathSum(length: Int, series: (Int) -> Int) -> Int
```

The first parameter, `length`, defines the number of values to sum. The second parameter, `series`, is a lambda that can be used to generate a series of values. `series` should have a parameter that is the position of the value in the series and return the value at that position.

`mathSum` should calculate `length` number of values, starting at position 1, and return their sum.

Use the function to find the sum of the first 10 square numbers, which equals 385. Then use the function to find the sum of the first 10 Fibonacci numbers, which equals 143.

For the Fibonacci numbers, you can use the function you wrote in the challenges of the functions chapter — or grab it from the solutions if you're unsure what you've done is correct.

Challenge 3: Functional ratings

In this final challenge, you will have a list of app names with associated ratings they've been given. Note — these are all fictional apps!

Create the data map like so:

```
val appRatings = mapOf(
  "Calendar Pro" to arrayOf(1, 5, 5, 4, 2, 1, 5, 4),
  "The Messenger" to arrayOf(5, 4, 2, 5, 4, 1, 1, 2),
  "Socialise" to arrayOf(2, 1, 2, 2, 1, 2, 4, 2)
)
```

First, create a map called `averageRatings` which will contain a mapping of app names to average ratings. Use `forEach` to iterate through the `appRatings` map, then use `reduce` to calculate the average rating and store this rating in the `averageRatings` map.

Finally, use `filter` and `map` chained together to get a list of the app names whose average rating is greater than 3.

Key points

- **Lambdas** are functions without names. They can be assigned to variables and passed as arguments to functions.
- Lambdas have **shorthand syntax** that makes them a lot easier to use than other functions.
- A lambda can **capture** the variables and constants from its surrounding context.
- A lambda can be used to direct how a collection is sorted.
- There exists a handy set of functions on collections which can be used to iterate over the collection and transform the collection. Transforms include mapping each element to a new value, filtering out certain values, and folding or reducing the collection down to a single value.

Where to go from here?

Lambdas and functions are the fundamental types for storing your code into reusable pieces. Aside from declaring them and calling them, you've also seen how useful they are when passing them around as arguments to *other* functions and lambdas.

That finishes this part of the book on "Collections & Lambdas". Next up, it's time to learn about creating your own types.

Section III: Building Your Own Types

You can create your own type by combining variables and functions into a new type definition. For example, integers and doubles might not be enough for your purposes, and you might need to create a type to store complex numbers. Or maybe storing first, middle and last names in three independent variables is getting difficult to manage, so you decide to create a `FullName` type.

When you create a new type, you give it a name; thus, these custom types are known as **named types**. Named types are a powerful tool for modeling real-world concepts. You can encapsulate related concepts, properties and methods into a single, cohesive model.

Custom types make it possible to build large and complex things with the basic building blocks that you've learned so far. It's time to take your Kotlin apprenticeship to the next level!